PRACTICAL NO. 6

AIM: Considered there are N philosophers seated around a circular table with one chopstick between each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pick up the two chopsticks adjacent to him. one chopstick may be picked up by any one of its adjacent followers but not both. Write a program to solve the problem using process synchronization technique.

CODE:

```
hari_ai@Harryfunlapcare: ~
GNU nano 7.2                                    dining2.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define N 5
#define MAX_EAT 3   // Each philosopher will eat only 3 times

sem_t chopstick[N];

void* philosopher(void* num) {
    int id = *(int*)num;

    for (int i = 0; i < MAX_EAT; i++) {

        printf("Philosopher %d is thinking\n", id);
        sleep(1);

        sem_wait(&chopstick[id]);   // Left chopstick
        sem_wait(&chopstick[(id + 1) % N]);  // Right chopstick

        printf("Philosopher %d is eating (%d time)\n", id, i+1);
        sleep(1);

        sem_post(&chopstick[id]);
        sem_post(&chopstick[(id + 1) % N]);

        printf("Philosopher %d finished eating\n", id);
    }

    printf("Philosopher %d left the table\n", id);
    pthread_exit(NULL);
}

int main() {
    pthread_t thread[N];

^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A Set Mark   M-] To Bracket
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line M-E Redo      M-6 Copy       ^Q Where Was
```

```
hari_ai@Harryfunlapcare: ~
GNU nano 7.2                                    dining2.c *
        sem_wait(&chopstick[(id + 1) % N]);  // Right chopstick

        printf("Philosopher %d is eating (%d time)\n", id, i+1);
        sleep(1);

        sem_post(&chopstick[id]);
        sem_post(&chopstick[(id + 1) % N]);

        printf("Philosopher %d finished eating\n", id);
    }

    printf("Philosopher %d left the table\n", id);
    pthread_exit(NULL);
}

int main() {
    pthread_t thread[N];
    int id[N];

    for (int i = 0; i < N; i++) {
        sem_init(&chopstick[i], 0, 1);
    }

    for (int i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&thread[i], NULL, philosopher, &id[i]);
    }

    for (int i = 0; i < N; i++) {
        pthread_join(thread[i], NULL);
    }

    printf("\nAll philosophers have finished eating.\n");

    return 0;
}

^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A Set Mark   M-] To Bracket
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line M-E Redo      M-6 Copy       ^Q Where Was
```

OUTPUT:

```
hari_ai@Harryfunlapcare: ~
hari_ai@Harryfunlapcare:~$ nano dining2.c
hari_ai@Harryfunlapcare:~$ gcc dining2.c -o dining2 -lpthread
hari_ai@Harryfunlapcare:~$ ./dining2
Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 3 is thinking
Philosopher 4 is thinking
Philosopher 2 is eating (1 time)
Philosopher 2 finished eating
Philosopher 2 is thinking
Philosopher 1 is eating (1 time)
Philosopher 1 finished eating
Philosopher 1 is thinking
Philosopher 0 is eating (1 time)
Philosopher 0 finished eating
Philosopher 0 is thinking
Philosopher 4 is eating (1 time)
Philosopher 4 finished eating
Philosopher 4 is thinking
Philosopher 3 is eating (1 time)
Philosopher 3 finished eating
Philosopher 3 is thinking
Philosopher 2 is eating (2 time)
Philosopher 2 finished eating
Philosopher 2 is thinking
Philosopher 1 is eating (2 time)
Philosopher 0 is eating (2 time)
Philosopher 1 finished eating
Philosopher 1 is thinking
Philosopher 0 finished eating
Philosopher 0 is thinking
Philosopher 4 is eating (2 time)
Philosopher 3 is eating (2 time)
Philosopher 4 finished eating
Philosopher 4 is thinking
Philosopher 3 finished eating
Philosopher 3 is thinking
Philosopher 2 is eating (3 time)
Philosopher 2 finished eating
Philosopher 2 left the table
```

```
hari_ai@Harryfunlapcare: ~
Philosopher 0 finished eating
Philosopher 0 is thinking
Philosopher 4 is eating (1 time)
Philosopher 4 finished eating
Philosopher 4 is thinking
Philosopher 3 is eating (1 time)
Philosopher 3 finished eating
Philosopher 3 is thinking
Philosopher 2 is eating (2 time)
Philosopher 2 finished eating
Philosopher 2 is thinking
Philosopher 1 is eating (2 time)
Philosopher 0 is eating (2 time)
Philosopher 1 finished eating
Philosopher 1 is thinking
Philosopher 0 finished eating
Philosopher 0 is thinking
Philosopher 4 is eating (2 time)
Philosopher 3 is eating (2 time)
Philosopher 4 finished eating
Philosopher 4 is thinking
Philosopher 3 finished eating
Philosopher 3 is thinking
Philosopher 2 is eating (3 time)
Philosopher 2 finished eating
Philosopher 2 left the table
Philosopher 1 is eating (3 time)
Philosopher 1 finished eating
Philosopher 1 left the table
Philosopher 0 is eating (3 time)
Philosopher 0 finished eating
Philosopher 0 left the table
Philosopher 4 is eating (3 time)
Philosopher 4 finished eating
Philosopher 4 left the table
Philosopher 3 is eating (3 time)
Philosopher 3 finished eating
Philosopher 3 left the table

All philosophers have finished eating.
hari_ai@Harryfunlapcare:~$
```